Assignment 1 : A Simple Paging Simulator

Topic: Memory management (paging, swapping, internal fragmentation, logical to physical address translation)

Due: 11:59 PM, Sunday, September 27, 2026 (Always check Canvas on updated due date) (Canvas → Assignment 1)

---

1. Learning objectives

After completing this assignment you should be able to:

1. Translate a job's byte size into a page count and reason about **internal fragmentation**.
2. Manage a frame table and per-process **page tables**, including the page-to-frame mapping.
3. Implement a **FIFO** page/job-replacement policy and a manual **suspend/resume** (swap out / swap in) mechanism.
4. Distinguish clearly between a job that is *resident*, *suspended (on secondary storage)*, and *removed (gone from the system)*.
5. Perform **logical-to-physical address translation** through a page table, including bounds and residency checks — the central purpose of paging.

---

2. Overview

You will simulate a **simple paging system**. Because this is *simple* paging, a job can execute only when **all** of its pages are resident in main memory; there is no demand paging. If main memory cannot hold a new (or resumed) job, you must swap other jobs out to secondary storage to make room, using the rules in Section 6.

Main memory is divided into equal-size **frames**. A job is divided into equal-size **pages** of the same size. You may assume the page size evenly divides the memory size, so the number of frames is always a whole number. A job's size, however, need **not** be a multiple of the page size — the leftover space in its last page is internal fragmentation.

---

3. Running the program

```
./page_system_simulator <memory_size_bytes> <page_size_bytes>
<initial_job_requests.txt>
```

On startup the program:

1. Divides memory into `frames = memory_size / page_size`.
2. Processes every line of the initial-request file **in order**, exactly as if each line had been typed interactively (Section 5).
3. Then reads further commands interactively from standard input until `exit`.

Numbering convention used throughout this spec and in the worked example: **frames and pages are numbered starting at 1.** You may use 0-based numbering internally, but state which convention your output uses and be consistent.

---

4. Input file format

A whitespace/tab-separated file. The first line is an optional header (`Job_ID  Size`) and must be ignored if present. Every other line is one command, in the form `<token> <value>`, or a single-token command (`print`, `exit`). Example:

```
Job_ID   Size
1        35000
2        4096
...
7        -1
3        -1
print
7        -2
print
```

Interactive input uses the same two-token form. (In the original prompt these were shown with a leading `$`; that was only a shell prompt and is **not** part of the input.)

---

5. Commands

Command	Meaning
`J bytes` (`bytes > 0`)	Admit a **new job** `J` of the given size. See admission rules in §6.3.
`J 0`	**Remove** job `J` from the system entirely (frees its frames if resident, discards its secondary-storage copy if suspended, deletes its page table).
`J -1`	**Suspend** job `J`: swap all its pages to secondary storage, free its frames, and keep its page table (marked as swapped out).
`J -2`	**Resume** suspended job `J`: bring it back into main memory, suspending other jobs first if necessary (§6.4).

Command	Meaning
`translate J a`	Translate **logical address** `a` (a byte offset into job `J`) into a physical address, printing the page number, offset, and frame. See §6.7.
`print`	Print a full snapshot of the system (§7).
`exit`	Terminate the simulator.

Error handling (required): for each of the following, print a short, clear message and take no other action. Do **not** crash.

- Admitting a job number that **already exists** in the system (resident *or* suspended) → reject as a duplicate.
- Admitting a job whose page count exceeds the total number of frames → reject as too large (§6.3).
- Removing, suspending, or resuming a job that does **not exist**.
- Suspending a job that is already suspended, or resuming a job that is **not** suspended (e.g. already resident).
- Translating an address for a nonexistent job, a suspended (non-resident) job, or an address outside the job's valid range (§6.7).

---

6. Core policies (precise definitions)

These are the rules that make your output reproducible. They are stated explicitly here because they were implicit in the original example.

6.1 Page count and internal fragmentation

For a job of size `S` bytes with page size `F`:

```
pages         = ceil(S / F)
internal_frag = pages * F - S        (all of it in the last page)
```

Internal fragmentation is a property of the job and does not depend on whether it is resident or suspended. Report it for every existing job (§7). Example: a 35000-byte job with `F = 4096` needs 9 pages and has `9*4096 - 35000 = 1864` bytes of internal fragmentation.

6.2 Frame allocation order

When a job is loaded (either newly admitted or resumed), assign its pages to the **lowest-numbered free frames**, in ascending frame order (first-fit by frame index). Page 1 goes to the lowest free frame, page 2 to the next, and so on.

This rule will help the printed layout to be the same for everyone.

6.3 Admitting a new job `J` of `bytes` (`> 0`)

1. If a job numbered `J` already exists (resident or suspended) → **reject (duplicate)**.
2. Compute `pages = ceil(bytes / F)`.
3. If `pages > total_frames` → **reject (too large for memory)**. Note this is the *only* case that permanently rejects for size: if it fits in an empty memory, it must be admitted, swapping others out if needed.
4. If free frames `>= pages` → load it (§6.2).
5. Otherwise → run the FIFO swap-out (§6.5) until free frames `>= pages`, then load it.

6.4 Resume `J -2`

`J` must currently be suspended (otherwise it is an error, §5).

1. Let `pages` be `J`'s page count.
2. If free frames `< pages`, suspend other **resident** jobs, oldest first by FIFO order (§6.5), one whole job at a time, until free frames `>= pages`.
3. Load `J` into the lowest-numbered free frames (§6.2). A resumed job counts as newly loaded for FIFO purposes (it goes to the tail of the queue).

Because a job's page count can never exceed the total number of frames, suspending enough other jobs always creates room — resume never fails once the job exists as suspended.

6.5 FIFO replacement / swap-out

Maintain a single FIFO queue of **resident** jobs, ordered by the time each job most recently entered main memory (initial load or resume appends to the tail; suspend/remove takes the job out of the queue).

When you must free space, evict from the **head** (the oldest resident job): swap **all** of that job's pages to secondary storage, free its frames, and mark it **suspended** (its page table is retained, showing pages as swapped out). Repeat with the next-oldest job until enough free frames exist. A job swapped out by this mechanism is in exactly the same *suspended* state as one suspended manually with `-1`.

6.6 Secondary storage

Assume secondary storage is unlimited; suspending/swapping out never fails for lack of disk space.

6.7 Address translation (`translate J a`)

A logical address `a` is a byte offset into job `J`, starting at 0. Translation succeeds only when the job exists **and is resident** (under simple paging a non-resident job cannot execute, so there is no physical address to compute). The steps:

```
page_index (0-based)          = a / page_size          (integer
division)
offset                        = a % page_size
page_number (printed, 1-based) = page_index + 1
frame_number                  = the frame holding that page (from
the page table)
physical_address              = (frame_number - 1) * page_size +
offset
```

Print `page_number`, `offset`, `frame_number`, and `physical_address` on success.

Reject with a clear message (no physical address) when:

1. Job `J` does not exist.
2. Job `J` is suspended (not resident).
3. `a` is outside the job's valid range, i.e. `a < 0` or `a >= job_size`.

Case 3 has a subtlety worth testing: an address can land inside a resident frame yet still be invalid, because the job's last page is only partly used (the internal-fragmentation region). For example, a 12000-byte job with a 4096-byte page occupies 3 pages spanning byte offsets 0–12287, but offsets 12000–12287 are past the job's real data and must be rejected even though the frame exists.

---

7. The `print` snapshot

Design a readable terminal layout, but it **must** contain all of the following. A suggested format is shown below so that outputs are comparable across submissions.

(a) Frame table — every frame from 1 to `N`, marked either *free* or with the job number and which page of that job occupies it.

(b) Page table for every existing job (resident and suspended) — for each job: its state (resident / suspended), size in bytes, page count, and a per-page mapping to either a frame number or "swapped out / on secondary storage", plus its internal fragmentation.

(c) Summary — number of free frames and total internal fragmentation across resident jobs.

Suggested layout:

```
===== MEMORY SNAPSHOT =====
Frame  1: Job 10, page 1        Frame  9: (free)
```

```
Frame  2: Job 10, page 2        Frame 10: Job 9,  page 1
...                             ...
Free frames: 8, 9, 11, 15, 16   (5 free)

--- Page tables ---
Job 10  [RESIDENT]  size=24000  pages=6  int.frag=576
  page 1 -> frame 1
  page 2 -> frame 2
  ...
Job 1   [SUSPENDED] size=35000  pages=9  int.frag=1864
  page 1 -> (secondary storage)
  ...

Total internal fragmentation (resident): ... bytes
==========================
```

---

8. Worked example (verify your implementation against this)

Run with `./page_system_simulator 65536 4096 initial_job_requests.txt`, giving `65536 / 4096 = 16` frames, using the example file in §4.

Step-by-step:

1. **Load jobs 1–8.** Job 1 needs `ceil(35000/4096)=9` pages (frames 1–9); jobs 2–8 need 1 page each (frames 10–16). Memory is now exactly full.
2. **Remove jobs 2, 4, 6, 8.** This frees frames 10, 12, 14, 16.
3. **Admit job 9** (`ceil(12000/4096)=3` pages). Four frames are free (10, 12, 14, 16); take the lowest three → frames 10, 12, 14. Frame 16 is left free.
4. **Admit job 10** (`ceil(24000/4096)=6` pages). Only frame 16 is free, so FIFO evicts the oldest resident job, **job 1** (frames 1–9), which is now suspended. Free frames are now 1–9 and 16; load job 10 into frames 1–6.
5. **Suspend job 7** (was frame 15) and **suspend job 3** (was frame 11).
6. **Resume job 7** (1 page). There is free space, so no further suspension is needed; job 7 loads into the lowest free frame → **frame 7**.

Final `print` should show:

- Frames 1–6 → job 10; frame 7 → job 7; frames 10, 12, 14 → job 9; frame 13 → job 5. Frames 8, 9, 11, 15, 16 are free.
- Page tables exist for jobs 1, 3, 5, 7, 9, 10. **Jobs 1 and 3 are suspended** (all pages on secondary storage). **Jobs 2, 4, 6, 8 are gone** — no page table.
- Internal fragmentation: job 1 = 1864, job 9 = 288, job 10 = 576, jobs 3/5/7 = 0.

Translation examples (from this final state, page size 4096):

- `translate 10 5000` → page 2, offset 904. Job 10's page 2 is in frame 2, so physical = `(2-1)*4096 + 904 =` **5000**.
- `translate 9 5000` → page 2, offset 904. Job 9's page 2 is in frame 12, so physical = `(12-1)*4096 + 904 =` **45960**.

  The same logical address (5000) resolves to two completely different physical addresses depending on the job — this contrast is the whole point of paging, and a good short-answer prompt.
- `translate 9 12100` → **error**: 12100 is beyond job 9's size (12000), even though it falls inside page 3 (which is resident in frame 14). This is the internal-fragmentation edge case from §6.7.
- `translate 1 100` → **error**: job 1 is suspended, so it has no physical address.

---

9. Required runs

Run your simulator on **both** provided input files and capture the full terminal output (including every `print`):

Input	`memory_size`	`page_size`	Frames
Input file 1	65536	4096	16
Input file 2	20000	1000	20

---

10. Deliverables

Submit to the Canvas Assignment 1 folder:

1. **Source code**, named `firstname-lastname-1.<ext>` (e.g. `john-doe-1.c`, `.cpp`, `.java`, or `.py`).
2. **One output file per input**, named `output-1.txt` and `output-2.txt`.

Allowed languages: C, C++, Java, Python, or any reasonable language. Your code must compile/run from the command line with the invocation in §3; include a one-line build/run note at the top of your source if it needs special steps.

---

11. Edge-case checklist (self-test before submitting)

- Duplicate job number rejected (including when the existing job is suspended).
- Job larger than total frames rejected; a job that fits only after swapping is admitted.

- Removing / suspending / resuming a nonexistent job is handled gracefully.
- Suspending an already-suspended job, and resuming a resident job, are handled gracefully.
- After removals, new jobs fill the lowest-numbered free frames.
- FIFO evicts whole jobs, oldest first, and evicted jobs become *suspended* (page tables retained), not *removed*.
- Internal fragmentation is correct for a job whose size is a multiple of the page size (it should be 0).
- Address translation gives the right page, offset, frame, and physical address for a mid-job address.
- Translation rejects an address that is past the job's size but still inside its last (resident) page.
- Translation rejects an address for a suspended job and for a nonexistent job.

---

12. Grading (100 points, This is a suggested rubric and TA or Instructor may change it if necessary)

Category	Points
Correct startup, argument/file parsing, frame count	8
New-job admission, page-count math, first-fit allocation	13
Remove / suspend / resume semantics	17
FIFO swap-out (correct victim selection, whole-job eviction)	17
Page tables + frame table + internal fragmentation in `print`	15
Address translation (page/offset/frame math, bounds and residency checks)	15
Error handling and edge cases (§11)	10
Output matches the worked example (§8) and required runs (§9)	5

---

13. Academic integrity

You may discuss concepts and your progress with classmates, but you must design and write your own code. **Do not use Gen AI to produce your output code**. Purpose of this assignment is for you to understand how simple paging works, and you need to do this by yourself to properly understand the concepts. When in doubt, ask the instructor.